

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ» (НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

Факультет _____

Кафедра _____

Направление подготовки _____

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

(Фамилия, Имя, Отчество автора)

Тема работы _____

«К защите допущена»

Заведующий кафедрой

ученая степень, звание

...../.....

(фамилия, И., О.) / (подпись, МП)

«.....».....20...г.

Научный руководитель

ученая степень, звание

должность, место работы

...../.....

(фамилия, И., О.) / (подпись, МП)

«.....».....20...г.

Дата защиты: «.....».....20...г.

Новосибирск, 20__

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1. Предметная область	6
1.1. Функциональные промежуточные представления	6
1.1.1. Программирование в стиле передачи продолжений .	7
1.2. Императивные промежуточные представления	7
1.2.1. CFG	7
1.2.2. SSA	7
1.2.3. More нод	8
1.3. Соответствие между CPS и SSA	8
1.4. Thoring IR	8
2. РАЗРАБОТАННЫЙ ПОДХОД	9
3. РЕЗУЛЬТАТЫ	10
ЗАКЛЮЧЕНИЕ	11
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	12
ПРИЛОЖЕНИЕ	13
П.1. Первая глава приложения	13

ВВЕДЕНИЕ

Современные языки программирования предоставляют уровень абстракции, позволяющий разрабатывать поддерживаемый и эффективный код в разумные временные сроки. Тем не менее, язык ассемблера целевой архитектуры таких абстракций не содержит. Отсюда возникает задача по переводу высокоуровневого языка в машинный код, которую решает программа, называемая *компилятором*. При этом “перевод” довольно точная аналогия для описания работы компилятора - недостаточно транслировать высокоуровневые конструкции “дословно”, такая программа попросту была бы неэффективной. Как и хороший переводчик, компилятор адаптирует (или привычнее сказать *оптимизирует*) исходную программу по мере её трансляции в целевой язык.

Такая трансляция производится в несколько этапов, как показано на рис. 0.1. На ранних стадиях компиляции происходит проверка принадлежности текста программы исходному языку. Параллельно с этим строится *представление* программы в виде *дерева абстрактного синтаксиса*[1]. Это наиболее удобное представление программы как *текста*, используемое для проведения синтаксического и семантического анализов.

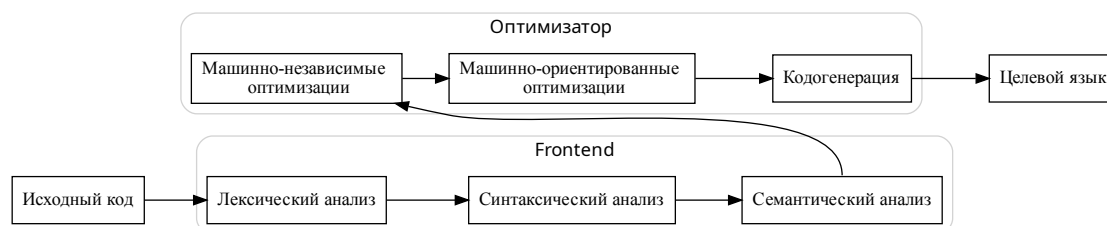


Рис. 0.1: Пример архитектуры компилятора

На следующем этапе компиляции производятся оптимизации программы, не требующие непосредственного отображения высокоуровневых инструкций в машинные. Такие преобразования называются *машинно-независимыми* оптимизациями. Абстрактное синтаксическое дерево исходной программы часто оказывается непригодным для проводимых в этой фазе анализов и трансформаций кода. Поэтому компилятор переводит АСД в более подходящее для оптимизаций представление, называемое *промежуточным представлением*. Как следствие это представление зависит от компилируемого языка.

Например, основными абстракциями императивных языков программирования являются *последовательное исполнение выражений*, использование *управляющих конструкций*, наличие *переменных*, разбиение программ на *процедуры* или *функции*. И промежуточное представление выбирается так, чтобы явно отражать эти концепции. Классическим примером является *граф потока управления*[2], такое промежуточное представление использует LLVM[3].

Языки функциональной парадигмы программирования, такие как Haskell[4], SML[5], Scheme[6], выбирают в качестве промежуточного представления различные модификации лямбда-исчисления, так как они наиболее полно отражают концепции исходного языка: *функциональные замыкания*, *функции высших порядков*, *алгебраические типы данных* и т.д.

Поздние фазы компиляции отвечают за проведение машинно-зависимых оптимизаций и кодогенерации. В процессе промежуточное представление *занижается* (от англ. *lowering*), высокоуровневые инструкции превращаются в набор низкоуровневых, отражающих семантику целевого языка в

большей степени, чем исходного. Такое промежуточное представление принято называть *низкоуровневым*.

Таким образом парадигма исходного языка напрямую влияет на построение оптимизатора. Современные языки программирования, однако, все более тяготеют к смешению парадигм, абстракции изначально появившиеся в функциональных языках перетекают и активно используются в современных объектно-ориентированных языках (самый яркий пример - функциональные замыкания). Однако *промежуточные представления* в основном остаются неизменными, и заимствованные концепции выражаются в терминах исходного языка[8], что приводит к ухудшению результатов оптимизаций.

Отсюда возникает идея реализации *мультипарадигменного промежуточного представления*. В этой работе представлены результаты разработки *функционального* расширения *императивного* промежуточного представления и анализ его эффективности в рамках экспериментального компилятора *ExcelsiorHuawei*.

1. Предметная область

1.1. Функциональные промежуточные представления

TODO

В качестве *промежуточного представлений* оптимизаторов функциональных языков программирования используются различные модификации *лямбда-исчисления*(1.1).

$$term ::= var \mid (term \ term) \mid (\lambda var. term) \quad (1.1)$$

Классические преобразования термов лямбда исчисления, такие как *β -редукция*, *η -конверсия* подходят для описания большинства оптимизаций функциональных языков. Посмотрим на терм $(\lambda x. 0) (f y)$, применение к нему шага β -редукции соответствует удалению избыточных вычислений¹.

Набор подобных преобразований термов в оптимизаторе называется *правилами переписывания* термов. Например правило для удаления лишних вычислений можно записать так

$$(\lambda x. M)N \rightarrow M \text{ } x \notin FV(M)$$

На практике в оптимизаторах часто используется *типизированное лямбда исчисление*, с добавленным связыванием переменных (англ. let-bindings) и сопоставлением с образцом (англ. pattern-matching)[4].

Для знакомства с функциональными *промежуточными представления-*

¹ Конечно, такая бета-редукция применима только к ленивым языкам, т.к. вычисление терма $(f y)$ может привести к исключению или к незавершающемуся исполнению.

ми нам будет достаточно классического *лямбда исчисления* со связыванием переменных.

$$(\lambda x. M)N \equiv \text{let } x = N \text{ in } M$$

1.1.1. Программирование в стиле передачи продолжений

TODO

Программирование в стиле передачи продолжений (англ. continuation passing style, CPS), это шаблон программирования, позволяющий сделать поток управления программы явным, используя для этого функции высшего порядка, которые мы будем называть *продолжением* (англ. *continuation*). Будем говорить, что функция находится в *cps*, если:

1. Ну принимает продолжение тип, ещё там тип есть $(t \rightarrow' a)$, где t - это известный тип, а $'a$ - некоторая типовая переменная.
2. Функция является хвостовой рекурсией.

1.2. Императивные промежуточные представления

1.2.1. CFG

TODO

1.2.2. SSA

TODO

1.2.3. Море нод

TODO

1.3. Соответствие между CPS и SSA

TODO Поможет при описании преобразования Thorin'a обратно в SSA

1.4. Thoring IR

TODO

2. РАЗРАБОТАННЫЙ ПОДХОД

TODO

3. РЕЗУЛЬТАТЫ

TODO

ЗАКЛЮЧЕНИЕ

TODO

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. W.Appel A. Modern Compiler Implementation in ML. Cambridge University Press, 1998.
2. Владимиров К. Оптимизирующие компиляторы. Структура и алгоритмы. Litres, 2024.
3. LLVM Project. LLVM Language Reference Manual. 2025.
4. Jones S.L.P., Santos A.M. A transformation-based optimiser for Haskell // Science of computer programming. Elsevier, 1998. Т. 32, № 1-3. С. 3–47.
5. Appel A., Jim T. Continuation-passing style, closure-passing style // 16th Ann. ACM Symp. on Principles of Programming Languages. 1989.
6. Adams N. и др. Orbit: An optimizing compiler for Scheme // ACM SIGPLAN Notices. ACM New York, NY, USA, 1986. Т. 21, № 7. С. 219–233.
7. Gosling J. и др. The Java Language Specification, Java SE 8 Edition. Addison-Wesley, 2015.
8. Evans B. Understanding Java method invocation with invokedynamic // Java Magazine. 2021.

ПРИЛОЖЕНИЕ

П.1. Первая глава приложения

TODO